

INCIDENT REPORT

Multi-Stage Intrusion on a Windows Endpoint

Reference IR-2026-0417 · Classification: High · Status: Contained

FIELD	DETAIL
Prepared by	Chayan Panchal — HTB Certified Defensive Security Analyst (CDSA)
Contact	chayanpanchal11@gmail.com · linkedin.com/in/chayanpanchal
Report date	24 September 2026
Affected asset	Single Windows endpoint (workstation)
Telemetry available	Sysmon, Windows Security event log. No EDR. No originating alert.
Tooling used	PowerShell Get-WinEvent, Windows Event Viewer
Incident type	Malware execution → process injection → credential access
Outcome	Five-stage attack chain reconstructed. Credential theft confirmed. No lateral movement observed.

Scope and provenance of this document

This investigation was carried out in a Hack The Box Academy lab environment, not a production network. It is written in client-deliverable format to demonstrate incident reporting practice.

Specific file names, hashes and host identifiers are withheld in line with HTB publication guidance. Indicators are therefore described behaviourally rather than by signature. All analysis, queries and conclusions are my own work.

1. Executive Summary

A Windows endpoint was reported as suspected compromised. No security alert had fired and no endpoint detection product was deployed, so the investigation began from raw event logs with no indication of where on the host to look.

The host was confirmed compromised. An attacker achieved code execution, concealed that execution inside a trusted Windows process, and successfully harvested credential material from memory. The full sequence was reconstructed across five distinct stages.

The finding that determined our response

Credential material was stolen, but it had not yet been used.

Every authentication event on the host after the moment of theft was a routine system service logon. No interactive or remote logon followed. The attacker held credentials but had not moved to another machine.

This distinction is the difference between isolating one workstation and treating the domain as compromised. It is why this report recommends targeted containment rather than a domain-wide credential reset.

Business impact

- Credentials belonging to any account with an active session on this host must be treated as compromised and reset.
- No evidence of access to other systems, data exfiltration, or service disruption was identified.
- Containment remained viable at the host level at the time of analysis. That window closes if the attacker retains access, which is why isolation is the first recommended action.

Priority recommendations

- Isolate the affected host from the network immediately, preserving it for forensic imaging.
- Force a credential reset for every account that held a session on the host, not for the whole domain.
- Deploy the four detections listed in Section 9, which would have surfaced this chain automatically.
- Enable and centrally collect Sysmon Event ID 11 (FileCreate), whose absence cost us the earliest point of detection.

2. Incident Classification and Severity

ATTRIBUTE	ASSESSMENT	BASIS
Severity	High	Credential access (T1003.001) was achieved against LSASS. Successful credential theft is treated as High regardless of subsequent use.
Confidence	High	Five independent telemetry sources corroborate a single actor. The staging directory observed at Stage 1 reappears at Stage 5, linking the chain end to end.
Scope	Single host	No authentication activity consistent with lateral movement was observed following credential access.
Status	Contained (recommended)	Containment actions in Section 8 were scoped on the basis of the Stage 4 negative finding.
Data exfiltration	Not observed	No evidence identified within available telemetry. Network telemetry was not available, so this cannot be stated conclusively.

Stated limitation

Network telemetry, proxy logs and NetFlow were not available in this environment. Absence of observed exfiltration therefore reflects the limits of the data set, not a positive finding that no exfiltration occurred. In production this gap would be closed before the incident was closed.

3. Scope and Methodology

Sysmon generates a high volume of events, the overwhelming majority of them benign. Reading the log chronologically from the start of the day would have meant processing thousands of routine events before reaching anything of interest.

Instead I worked from the event types that carry the highest signal per unit of volume, establishing a foothold in the data set first and expanding outward from it.

SYSMON EVENT	CAPTURES	WHY PRIORITISED
7 — ImageLoaded	Every DLL load	Search-order hijacks surface as path anomalies against a known-good baseline
8 — CreateRemoteThread	Cross-process thread creation	Extremely low legitimate baseline. In practice almost always injection
10 — ProcessAccess	Handle opened to another process	The only reliable view of credential dumping without EDR

SYSMON EVENT	CAPTURES	WHY PRIORITISED
1 — ProcessCreate	Process creation with parent lineage	Parent/child relationships expose spoofed ancestry

Event ID 8 was queried first for exactly this reason: it is cheap to filter, rarely fires legitimately, and when it does fire it points directly at both the injector and the victim process.

4. Reconstructed Attack Chain

Five stages were identified. Each is presented with the evidence that established it and the reasoning applied, so that the conclusion can be independently reviewed.

Stage 1 — DLL search-order hijack (T1574.001)

Evidence: Sysmon Event ID 7 (ImageLoaded).

A trusted, Microsoft-signed Windows executable was observed loading a DLL from a user-writable directory under C:\ProgramData rather than from System32, where the legitimate library resides. Three characteristics together removed any ambiguity:

- Wrong directory. The legitimate DLL ships in System32. The observed path is user-writable, which is precisely why an attacker selects it.
- Placeholder version metadata. FileVersion read 0.0.0.0. Microsoft-shipped binaries always carry a genuine version string.
- Absent company name. The field was blank, where a signed Microsoft binary would be populated.

Any single indicator might be explained away. In combination they describe a hand-built payload placed where the Windows DLL search order would locate it before the genuine library. This established both the initial execution vector and the staging directory, which proved significant at Stage 5.

Stage 2 — Unmanaged PowerShell via CLR injection (T1059.001)

Evidence: Sysmon Event ID 7, queried against a different hypothesis.

clr.dll is the .NET Common Language Runtime and loads whenever managed code executes. The hunt was therefore for a process with no legitimate reason to run .NET code that had nonetheless loaded it.

Such a process indicates unmanaged PowerShell: shellcode that hosts the CLR inside an arbitrary process in order to execute PowerShell-equivalent code without ever launching powershell.exe. This technique defeats every detection built around monitoring that process name, which accounts for a substantial share of naive PowerShell detection logic.

Detection engineering implication

Detections keyed to a process name are brittle. The same capability, reached through a different host process, walks straight past them.

Detections keyed to capability — an unexpected process loading the runtime that capability requires — survive this evasion. This principle shaped the rules referenced in Section 9.

Stage 3 — Process injection (T1055)

Evidence: Sysmon Event ID 8 (CreateRemoteThread).

With the victim process identified at Stage 2, filtering Event ID 8 on that process as TargetImage surfaced the injecting process directly.

The injector was a legitimate, signed Windows utility present on every Windows installation — a Living-off-the-Land Binary. Its presence and execution are unremarkable in isolation. What is not unremarkable is that binary creating a thread inside another running process. This is the pivot on which the whole reconstruction turns: Event ID 8 links attacker-controlled code back to whatever placed it there.

Stage 4 — LSASS credential access (T1003.001)

Evidence: Sysmon Event ID 10 (ProcessAccess), followed by Security Event ID 4624.

LSASS holds credential material in memory. Any process opening a handle to it with read access is either legitimate endpoint protection or credential theft; there is very little middle ground. Filtering Event ID 10 on lsass.exe as TargetImage surfaced the responsible tool.

The determining question for incident severity is not whether credentials were dumped but whether they were subsequently used. Stolen credentials sitting unused are a containable problem. Stolen credentials already used for lateral movement mean the blast radius extends beyond the host.

Every Event ID 4624 recorded after the dump timestamp was a routine SYSTEM service logon. No interactive (Type 2),

network (Type 3) or remote interactive (Type 10) logon followed the credential theft. The attacker possessed credentials but had not yet moved.

That negative finding is as operationally valuable as any positive one. It is the entire basis for the containment scoping in Section 8.

Stage 5 — Parent PID spoofing and discovery (T1134.004, T1033)

Evidence: Sysmon Event ID 1 (ProcessCreate).

A Windows error-reporting utility was observed spawning cmd.exe, which in turn executed whoami. Error-reporting utilities do not open interactive shells. This is parent PID spoofing: the attacker set a trusted process as the parent of their payload so that an analyst scanning the process tree sees a benign lineage.

The corroborating tell was the whoami command itself — basic discovery, and characteristically the first action an operator takes after establishing execution in order to learn what privileges they landed with. The working directory tied the process back to the same C:\ProgramData staging path identified at Stage 1, closing the loop and confirming a single actor across all five stages rather than five unrelated findings.

5. MITRE ATT&CK Mapping

STAGE	TACTIC	TECHNIQUE	ID
1	Persistence / Defence Evasion	Hijack Execution Flow: DLL Search Order Hijacking	T1574.001
2	Execution	Command and Scripting Interpreter: PowerShell	T1059.001
3	Defence Evasion / Privilege Escalation	Process Injection	T1055
4	Credential Access	OS Credential Dumping: LSASS Memory	T1003.001
5	Defence Evasion	Access Token Manipulation: Parent PID Spoofing	T1134.004
5	Discovery	System Owner / User Discovery	T1033

6. Indicators of Compromise

In line with the provenance note on the cover page, specific file names, hashes and host identifiers from the lab environment are withheld. The indicators below are behavioural, which in practice makes them more durable than signatures: an attacker can trivially change a file name, but cannot change the fact that the technique requires these actions.

TYPE	INDICATOR	STAGE
Host — path	DLL loaded from a user-writable directory under C:\ProgramData by a Microsoft-signed binary	1
Host — metadata	Loaded module reporting FileVersion 0.0.0.0 with an empty CompanyName field	1
Behaviour	clr.dll loaded by a process with no legitimate managed-code function	2
Behaviour	Signed Living-off-the-Land Binary creating a remote thread in an unrelated process	3
Behaviour	Non-security process opening a handle to lsass.exe with read access	4
Behaviour	Windows error-reporting utility recorded as the parent of cmd.exe	5
Behaviour	whoami executed from a working directory under C:\ProgramData	5

7. Containment Scoping Decision

This section records not only what was recommended but why the narrower option was chosen, since that reasoning is the part most likely to be challenged in review.

Recommendation: targeted containment, not domain-wide credential reset

Isolate the affected host from the network, preserving volatile state for imaging.
Force a password reset for every account that held a session on this host during the incident window.
Do not initiate a domain-wide credential reset at this time.

Reasoning

A domain-wide credential reset is disruptive, costly, and resented by the business. It is the correct call when an attacker may already be authenticating as other users elsewhere. It is the wrong call when the evidence positively shows they are not.

Here the Stage 4 analysis established that no interactive or remote authentication followed the credential theft. The credentials at risk are therefore bounded by the set of accounts with sessions on this single host. Resetting the domain would impose a large outage to address a risk the telemetry does not support.

Getting that scoping right is the difference between a two-hour incident and a two-day outage.

Conditions that would change this recommendation

- Discovery of authentication events on other hosts using any account sessioned on the affected machine.
- Evidence that a domain administrator or service account with broad rights held a session on the host.
- Any indication that the credential material left the host — which, absent network telemetry, cannot currently be ruled out.
- Discovery of persistence mechanisms beyond those identified, suggesting the reconstruction is incomplete.

8. Eradication and Recovery

#	ACTION	RATIONALE
1	Isolate host, preserve for imaging	Prevents use of retained access; preserves evidence before eradication destroys it
2	Reset credentials for sessioned accounts	Scoped per Section 7. Stolen material is assumed usable indefinitely until rotated
3	Remove hijacking DLL from the staging directory	Eliminates the initial execution vector
4	Rebuild rather than clean the host	Injection and LoLBin abuse leave limited artefacts. Rebuild is the only reliable assurance
5	Restrict write access to the staging directory	Removes the precondition the hijack depended on
6	Hunt the same indicators fleet-wide	A single compromised host is a hypothesis about the fleet, not a conclusion
7	Monitor affected accounts post-reset	Detects continued attempts to use credentials that are no longer valid

9. Detection Gaps and Recommendations

This incident required manual reconstruction because nothing detected it. The following rules were written directly from this investigation and are published, with documented false-positive profiles and tuning notes, at github.com/akatetron/signal-over-noise.

DETECTION	WOULD HAVE FIRED AT	TECHNIQUE
DLL loaded from non-standard path	Stage 1 — earliest available detection	T1574.001
Remote thread created by LoLBin	Stage 3	T1055
LSASS handle opened by unexpected process	Stage 4	T1003.001
Process with implausible parent lineage	Stage 5	T1134.004

Deployed together, these convert a manual five-stage reconstruction into four alerts arriving in chronological order, with the earliest at the point of initial execution.

Telemetry gaps identified

- Sysmon Event ID 11 (FileCreate) was not available. It would have recorded the hijacking DLL being written to the staging directory before it was ever loaded — a full stage earlier than the earliest detection above. This is the single highest-value collection improvement available.
- Command-line logging on Event ID 1 was essential to Stage 5 and should be confirmed as enabled fleet-wide rather than assumed.
- No network telemetry was available. Its absence is why exfiltration cannot be ruled out, and why the incident cannot responsibly be closed on the current data set.

10. Appendix A — Queries

Queries are reproduced as executed, using native tooling only. No commercial product was available in this environment.

A.1 Stage 1 — modules loaded from the staging directory

```
Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" |  
  Where-Object { $_.Id -eq 7 -and $_.Message -match "ProgramData" } |  
  Select-Object TimeCreated, Message
```

A.2 Stage 3 — cross-process thread creation

```
Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" |  
  Where-Object { $_.Id -eq 8 } |  
  Select-Object TimeCreated, @{n='Detail';e={$_.Message}} |  
  Format-List
```

A.3 Stage 4 — authentication activity following credential access

```
Get-WinEvent -FilterHashtable @{LogName='Security'; ID=4624} |  
  Where-Object { $_.TimeCreated -gt $dumpTime } |  
  Select-Object TimeCreated, @{n='LogonType';e={$_.Properties[8].Value}}
```

Logon type values are interpreted as: 2 interactive, 3 network, 4 batch, 5 service, 10 remote interactive. Only type 5 was present after the dump timestamp.

11. Appendix B — Analyst Note

The most consequential output of this investigation was not the identification of the attack chain. It was the Stage 4 negative result — establishing what the attacker had not yet done.

Reconstructing the chain determines what happened. Establishing the absence of lateral movement determines what the organisation is asked to do about it, and at what cost. An analyst who reports only the former hands the containment decision to someone else.